U. S. Naval Research Laboratory
Washington, DC 20375-5320

NRL/6920/MR--2026/3

Driving Optical Simulation with a Coding Agent: Experience Operating Zemax OpticStudio Through the ZOS-API

ZOEY S. DAVIDSON

Interfacing Biology Branch
Center for Biomolecular Science and Engineering

September 23, 2026

Distribution Statement A. Approved for public release; distribution is unlimited.

This page intentionally left blank.

REPORT DOCUMENTATION PAGE

PLEASE DO NOT RETURN YOUR FORM TO THE ABOVE ORGANIZATION

1. REPORT DATE	2. REPORT TYPE	3. DATES COVERED	
September 23, 2026	NRL Memorandum Report	START DATE March 18, 2026	END DATE August 19, 2026

4. TITLE AND SUBTITLE
Driving Optical Simulation With a Coding Agent: Experience Operating Zemax OpticStudio Through the ZOS-API

5a. CONTRACT NUMBER	5b. GRANT NUMBER	5c. PROGRAM ELEMENT NUMBER
5d. PROJECT NUMBER	**5e. TASK NUMBER**	**5f. WORK UNIT NUMBER**
		21A1N5

6. AUTHOR(S)
Zoey S. Davidson

7. PERFORMING ORGANIZATION / AFFILIATION NAME(S) AND ADDRESS(ES)
Interfacing Biology Branch
Center for Biomolecular Science and Engineering
U.S. Naval Research Laboratory
4555 Overlook Avenue, SW
Washington, DC 20375

8. PERFORMING ORGANIZATION REPORT NUMBER
NRL/6920/MR--2026/3

9. SPONSORING / MONITORING AGENCY NAME(S) AND ADDRESS(ES)	10. SPONSOR / MONITOR'S ACRONYM(S) NUMBER	11. SPONSOR / MONITOR'S REPORT NUMBER(S)
Office of Naval Research One Liberty Center 875 N. Randolph Street, Suite 1425 Arlington, VA 22203-1995	ONR	

12. DISTRIBUTION / AVAILABILITY STATEMENT
DISTRIBUTION STATEMENT A: Approved for public release; distribution is unlimited.

13. SUPPLEMENTAL NOTES

14. ABSTRACT
We report practical experience using a large-language-model coding agent to design and analyze optical systems in Zemax OpticStudio. Across three projects—a spoiled corner-cube retroreflector, a short-throw projector, and a reproduction of an optical datacenter link—we find that agentic operation is not a substitute for optical-engineering knowledge but a substantial accelerator that lowers the cost of learning and exercising the software's many features. We describe an open, agent-facing "skill" that grounds generated code in the vendor help files and validates it against OpticStudio directly. We describe the workflow patterns (runner experiments driven by a markup language, smoke tests, static namespace checks, and human review in the graphical user interface) and pitfalls that most affected iteration speed. We argue that pairing verified physical simulators with agents lets resource-limited scientists test, design, and discard ideas faster and further afield than they otherwise could, and we invite community improvement of the shared tooling.

15. SUBJECT TERMS

16. SECURITY CLASSIFICATION OF:			17. LIMITATION OF ABSTRACT	18. NUMBER OF PAGES
a. REPORT	b. ABSTRACT	c. THIS PAGE		
U	U	U	SAR	7

19a. NAME OF RESPONSIBLE PERSON	19b. PHONE NUMBER *(Include area code)*
Zoey S. Davidson	(202) 404-1923

This page intentionally left blank.

CONTENTS

1. INTRODUCTION 1
2. AGENT-FACING TOOLING 1
3. CASE STUDIES 2
   3.1 Spoiled corner-cube retroreflector 2
   3.2 Short-throw projector 2
   3.3 Wide-and-slow microLED link 3
4. LESSONS AND PITFALLS 3
5. OUTLOOK 3

ACKNOWLEDGEMENTS 4
REFERENCES 4

This page intentionally left blank.

Driving optical simulation with a coding agent: experience operating Zemax OpticStudio through the ZOS-API

Z. S. Davidson[1]

[1] *U.S. Naval Research Laboratory, Washington, D.C. 20375*

(Dated: August 28, 2026)

We report practical experience using a large-language-model coding agent to design and analyze optical systems in Zemax OpticStudio. Across three projects—a spoiled corner-cube retroreflector, a short-throw projector, and a reproduction of an optical datacenter link—we find that agentic operation is not a substitute for optical-engineering knowledge but a substantial accelerator that lowers the cost of learning and exercising the software's many features. We describe an open, agent-facing "skill" that grounds generated code in the vendor help files and validates it against OpticStudio directly. We describe the workflow patterns (runner experiments driven by a markup language, smoke tests, static namespace checks, and human review in the graphical user interface) and pitfalls that most affected iteration speed. We argue that pairing verified physical simulators with agents lets resource-limited scientists test, design, and discard ideas faster and further afield than they otherwise could, and we invite community improvement of the shared tooling.

INTRODUCTION

Optical design software such as Zemax OpticStudio exposes a scripting interface, the Zemax OpticStudio Application Programming Interface (ZOS-API), that allows a program to build systems, run analyses, and read results without the graphical user interface (GUI). The API is powerful but sprawling, and—critically—its documentation is written for human engineers, not for autonomous agents: there is no machine-oriented specification, naming conventions vary across API namespaces, and many object properties are discoverable only at runtime. These are exactly the conditions under which a large-language-model (LLM) coding agent tends to hallucinate plausible-but-wrong calls.

Here we report on several months of driving OpticStudio through the ZOS-API with a coding agent (primarily Claude Code). In the process, we developed a reusable *skill* that instructs the agent on usage of common patterns in programming with the ZOS-API. Our central claim is that agentic operation supplements the optical engineer's workflow and markedly reduces the burden of learning the enormous feature set of Zemax OpticStudio. However, agentic operation of OpticStudio does not (today) replace optical engineering. An engineer who can specify the criteria that matter—distortion, dispersion, telecentricity, étendue, manufacturability—gains a tireless assistant; an engineer who cannot may be led confidently down a wrong path. We support this with three projects of increasing complexity and then generalize the workflow lessons.

This effort is one instance of a broader movement to drive engineering-simulation software with LLM agents. Multi-agent frameworks now automate computational fluid dynamics in OpenFOAM [1] and finite-element analysis in Abaqus [2, 3], and the major vendors have begun shipping their own assistants for Ansys [4], COMSOL [5], and MATLAB/Simulink [6]. General-purpose "skill" libraries package comparable capabilities across scientific domains [7], and agents have even been set loose to run and improve their own training experiments [8]. Recently, Anthropic composed an extensive prompt to demonstrate Claude orchestrating multiple open-source protein design models [9]. Recurring ingredients across these efforts—tool documentation, an abstraction layer (e.g. Model Context Protocol or MCP), and iterative verification—mirror what we found necessary. For OpticStudio specifically, the Python bridge ZOSPy [10] exposes the ZOS-API to scripting, and a recent in development MCP server [11] connects it to chat assistants. Our contribution is complementary: rather than a natural-language front end, we focus on the grounding-and-verification tooling that makes an agent's generated ZOS-API code reliable, and on documenting the workflow and pitfalls across three real optical-design projects.

AGENT-FACING TOOLING

We found that reliable agent operation on tasks such as placing lenses, configuring ray traces, or measuring distortions, depended less on the model (Claude Opus $\geq$4.6 or GPT-5.4) than on a small set of instructions. These instructions included scripts to verify generated ZOS-API code and directed the agent to read relevant documentation before implementing ZOS-API code. The instructions and scripts are packaged as an open skill, `zemax-zos-api`, comprising a concise entry document, eighteen topic references (connection modes, sequential and non-sequential design, optimization, physical-optics propagation, data extraction, and pythonnet pitfalls), and three helper scripts. The ZOS-API usage patterns were derived from extensive runtime probing of v24.1 rather than from the manual alone.

Three mechanisms mattered most. First, *help-file grounding*: a search-and-read utility performs hybrid se-

Manuscript approved September 18, 2026.

mantic and keyword search over the bundled OpticStudio and ZOS-API help files, so the agent's first response to an exception is to consult the documentation rather than to hallucinate (again). The agent occasionally needs an explicit reminder to search documentation. Second, a *static namespace checker* parses generated Python with the abstract syntax trees (`ast`) module, extracts ZOS-API attributes, and resolves each against an OpticStudio instance before executing the program. In one commit it caught fourteen broken chains introduced by a close naming collision between optimization tools and surface types (e.g., `EvenAsphere`→`EvenAspheric`). Third, *runtime property probes* list valid OpticStudio object settings which prevents the agent from creating invalid attributes that do not cause pythonnet to raise an exception. Together with smoke tests—running a merit function for a single cycle to confirm it is nonzero and that variables actually move it before committing to a long optimization—these tools converted most agent errors from silent physical nonsense into fast, localized failures. While these practices reduced agent token expenditure in completing tasks, user GUI inspection of completed optical system models served as the final check on correctness and usefulness of the design.

CASE STUDIES

In each of the following studies, designs and planning were initiated with only high-level target goals. Separate research efforts not described here benefited from modeling or designing these systems. Each study followed a common iterative pattern: gathering requirements and literature (research papers, textbooks, or patents), refining novel design concepts through discussions with colleagues and agentic exploration, and finally establishing an initial project directory structure with AI-assisted prompts to begin writing ZOS-API code. The goals of each project co-evolved with understanding the optical simulation results and the maturation of agent-driven OpticStudio techniques.

Spoiled corner-cube retroreflector

A perfect corner cube returns light to a near-point at the source; many applications instead need a controlled, broadened return spot. The agent built a two-phase tool: an analytical stage (pure Python) sizes the required dihedral-angle perturbation δ from beam divergence, diffraction, and the target return diameter, and a numerical stage constructs a non-sequential model via the ZOS-API. Three stackable spoiling mechanisms were modeled—dihedral-angle offsets that produce the classic six-lobe return, bowed (finite-radius) mirror faces, and single-face diffuse scatter—and evaluated with polarized traces of $\gtrsim 10^6$ rays into a detector co-located with the source. The baseline spoiled build reproduced the expected six-lobe pattern with an exact central null. Several implementation challenges provided useful lessons: OpticStudio's built-in corner-cube object lacks a dihedral-offset adjustment (requiring a custom three-mirror assembly), surface scatter settings were silently ignored when applied directly to face data, and generating publication-ready figures required rendering plots externally with Matplotlib rather than exporting them from OpticStudio. Each challenge provided an opportunity to refine the skill, reducing agent token expenditure when encountering similar tasks in subsequent projects.

Short-throw projector

The second project designs a short-throw projector around a transmissive spatial light modulator, imaging a 20×20 mm panel onto a substrate 14–30 cm away. The design was deliberately restricted to commercial off-the-shelf (COTS) optics and vendor "black-box" models; the availability of such files was itself a practical design constraint. Two illumination front ends are carried in parallel: an incoherent light-emitting diode (LED) path through a mixing-rod homogenizer, and a coherent diode-pumped solid-state (DPSS) laser path through relays and a refractive top-hat beam shaper (a πShaper). The split between simulation regimes was deliberate—coherent illumination was modeled sequentially with damped-least-squares optimization on lens spacings, while incoherent illumination and the full imaging chain used non-sequential traces with extended sources and detector-uniformity maps, cross-checked by a sequential model reporting modulation transfer function (MTF), telecentricity, and field curvature. Concrete outcomes included plane-by-plane light budgets (e.g., 24.4% vs. 16.7% LED-to-substrate throughput for two relay choices) and an MTF recovery of the worst field from 0.046 to 0.293 by stopping down a Fourier aperture.

A subtle pitfall recurred: collimated planes reported spuriously large wavefront error until a focus reference was removed. The first time studying reflected wavefront error (RWE), the agent concluded that tens of wavelengths was the best achievable for the optical system. But, the agent had not known or realized that there was an afocal image-space mode setting needed for measuring collimated RWE. User inspection of the built .zos file in the GUI environment revealed the issue. The final RWE measurement was better than $\lambda/10$. This correction and others like it form the bulk of notes described in the skill.

Wide-and-slow microLED link

The third project reproduces and extends MOSAIC [12], a wide-and-slow optical datacenter interconnect that couples hundreds of directly modulated microLEDs through a multicore imaging fiber to a complementary metal-oxide-semiconductor (CMOS) detector array. Because the original model was not published, the agent reconstructed it in non-sequential mode—microLEDs as cosine-power elliptical sources activated one at a time to fill a channel matrix—and validated coupling and crosstalk before extending toward higher density. A five-stage ladder progressed from a single link (energy conservation checked against the analytical Lambertian view factor to within 0.35%) to a realistic multicore bundle with extramural absorption and an optimized total-internal-reflection microlens. The implementation reproduced the paper's qualitative claim—crosstalk moving from +9.2 dB in free space to −16.5 dB with the absorbing multicore fiber—while the extension surfaced a sobering correction: replacing an idealized $\pm 12^\circ$ emission cone with a real, pitch-limited collimator—restricted to a standard manufacturing process and common refractive-index materials—cut coupling gain from 3.85× to 2.1× and regressed crosstalk to −8.2 dB. This is precisely the kind of étendue-limited reality check that makes a verified simulator, rather than an LLM's internal reasoning, indispensable. It is also a small step toward recursive-self improvement in the hardware frontier AI models run on.

LESSONS AND PITFALLS

Several practices consistently improved outcomes. *Human review in the GUI* was the single strongest debugging aid: visually verifiable results tightened the development loop and exposed bugs (buried elements, duplicated detector reports) that passed numerical checks. *Runner-style experiments*, in which one agent writes framework code that consumes a markup-language description and another agent (or the same agent after context compaction) writes and reasons about that description, improved the agent's physical reasoning and made runs reproducible with explicit pass/fail gates. *Version control and context hygiene*—git for backup and task delineation, compaction between tasks, and lightweight knowledge bases and to-do lists—reduced rediscovery and kept the human aware of what the agent was doing. For traceability, we insisted the agent commit analytical estimates to runnable, human-readable code rather than asserting numbers from memory. Even for standard optical modeling tasks, enforcing a structured workflow and explicit traceability for physical constants prevents unmaintainable code that complicates later verification or modifications. Ultimately, pairing structured workflows with the `zemax-zos-api` skill reduces both token consumption and iteration time.

Limitations remain. Long optimizations are best launched by the human: we found no clean way to pause and checkpoint an optimization step, which can run for a long time, so the agent prepares the system and merit function and the user runs it. Running a single optimization step first verified behavior before committing to longer runs. OpticStudio is computationally intensive, so adequate processing power and memory are critical for agent throughput. Most importantly, the skill and these examples were built from essentially zero prior OpticStudio knowledge; existing optical expertise would likely improve results further, even as the agent's ability to read the manual and give precise GUI instructions made the basics approachable and, frankly, more fun to learn.

OUTLOOK

Recent writing posits that validation is a major bottleneck in AI-driven science [13]. Validation of novel scientific concepts was already a bottleneck in technology development well before the introduction of AI agents [14]. Simulation tools provide a partial escape hatch, and optical systems are a particularly rigorous domain because they admit strong analytical checks and physically verified numerical tests. Reproducing older Zemax-based studies proved both feasible and, often, a springboard to improvements—a modest form of "auto-research" [8] in which an agent re-derives and then extends prior work. Without overstating the case, pairing verified physical simulators with agentic operation lets scientists with limited resources test, design, and discard ideas faster and further afield than they otherwise would.

Looking ahead, a verified simulator may also serve as a cheap and correct reward signal for training. The same checks that localized agent errors here—static namespace resolution, smoke tests, and energy-conservation or analytical cross-checks—constitute a graded verifier that can score generated code without human labels. This inverts the principal cost of a recent study in which Bridgewater and Thinking Machines Lab fine-tuned an open-weight model (Qwen3-235B) to replicate expert financial judgment [15]: their bottleneck was expensive expert annotation, whereas physics supplies our labels for free. A natural next step is therefore to distill the trajectories generated here[18]—with the simulator as reward—into a smaller, specialized model that drives the ZOS-API at lower cost and, plausibly, with fewer errors than a general frontier model. This distill-to-specialize path is not merely hypothetical: a concurrent agent for Abaqus scripting was distilled into a fine-tuned, lightweight open-weight model that recovered much of the full agent's competence on finite-element tasks [2]. Their finding that such a fine-tuned model beat every frontier model tested

on both accuracy and cost, while frontier increments (including the move from Claude Opus 4.6 to 4.8) barely moved these judgment tasks, suggests the leverage lies in domain-specific learning rather than in model scale alone.

Two cautions temper this. First, a fine-tuned driver inherits only what the verifier can score—API mechanics and physical validity—not the optical judgment (which criteria matter, and when an apparent defect is an artifact) that an engineer still supplies. Second, Bridgewater's own result, that prompt engineering plateaued while learning from data prevailed, is a small instance of Sutton's "bitter lesson": general methods that leverage computation tend to overtake hand-built human knowledge [16].

This cautions against overinvesting in bespoke scaffolding like ours, much of which more capable models may eventually absorb. However, it is not guaranteed that learning will capture the tacit, hardest-to-articulate judgment that defines expert design; where it does not, a verified simulator kept in the loop remains a safeguard. Moreover, the figure of merit for a resource-limited laboratory or commercial venture is capability per dollar: even if a larger general model eventually leads on accuracy, the economics can still favor a specialized one—Bridgewater's fine-tune was roughly fourteen times cheaper per task—so specialization may persist as the cost-efficient choice long after it ceases to be the most capable.

We are releasing the `zemax-zos-api` skill openly [17], with the caveat that it remains a small, actively maintained tool that will benefit from cleanup, and we invite the community to extend it. Minimal working Python ZOS-API examples of the three projects described here are also included. We expect that better agent-facing documentation for simulation tools generally—not just Zemax—will be a decisive factor in how quickly this mode of work spreads.

Funding for this work was provided by the Office of Naval Research. The projects and tooling described here were developed with the Claude Code and Codex agents, with design assistance from Gemini; all three tools also assisted with copyediting this manuscript. Thanks to Christopher Spillmann for discussion and edits. Views expressed are the author's own.

[1] L. Yue, N. Somasekharan, T. Zhang, Y. Cao, and S. Pan, "Foam-Agent 2.0: An End-to-End Composable Multi-Agent Framework for Automating CFD Simulation in OpenFOAM," arXiv:2509.18178 (2025), `10.48550/arXiv.2509.18178`.

[2] Z. Li, H. Ge, G. Tang, and Y. Liu, "ModSolAgent: Automated Finite Element Code Generation for Abaqus via LLM-Based Agent," *IEEE Trans. Ind. Informat.* (2026), `10.1109/TII.2026.3669495`.

[3] LIRAM-LIN, `AbaqusAgent`: a multi-agent LLM framework for Abaqus finite-element simulation (2026). GitHub repository, `https://github.com/LIRAM-LIN/AbaqusAgent`.

[4] Ansys, "Work Faster and Smarter With Ansys Engineering Copilot" (2024), `https://www.ansys.com/blog/work-faster-smarter-with-ansys-engineering-copilot`.

[5] COMSOL, "AI Copilot for Automating Simulation Setup in COMSOL Multiphysics," COMSOL Conference paper (2024), `https://www.comsol.com/paper/ai-copilot-for-automating-simulation-setup-in-comsol-multiphysics-145242`.

[6] MathWorks, "Guide to Agentic AI with MATLAB and Simulink" (2025), `https://www.mathworks.com/campaigns/offers/agentic-ai-with-matlab.html`.

[7] K-Dense AI, `scientific-agent-skills`: an agent-skills library for science (2026). GitHub repository, `https://github.com/K-Dense-AI/scientific-agent-skills`.

[8] A. Karpathy, `autoresearch`: AI agents running research on single-GPU nanochat training automatically (2026). GitHub repository, `https://github.com/karpathy/autoresearch`.

[9] A. Shanehsazzadeh, Autonomous de novo protein binder design with Claude (2026). `https://www.anthropic.com/research/Claude-accelerates-protein-design`.

[10] L. van Vught, C. Haasjes, and J.-W. M. Beenakker, "ZOSPy: optical ray tracing in Python through OpticStudio," *J. Open Source Softw.* **9**(96), 5756 (2024), `10.21105/joss.05756`.

[11] J. A. Ruiz, `OpticStudioMCPServer`: a Model Context Protocol server for Zemax OpticStudio (2026). GitHub repository, `https://github.com/jaruiz6363/OpticStudioMCPServer`.

[12] A. Benyahya *et al.*, "MOSAIC: Breaking the Optics versus Copper Trade-off with a Wide-and-Slow Architecture and MicroLEDs," in *Proc. ACM SIGCOMM* (2025), `10.1145/3718958.3750510`.

[13] D. Wallace, C. Griffin, S. O'Neill, T. Luong, O. Larter, `Conjecture Machines`: AI agents and the new validation bottleneck in science (2026). Google DeepMind, `https://deepmind.google/public-policy/conjecture-machines-ai-agents-and-the-new-validation-bottleneck-in-science/`.

[14] V. Bush. "As we may think." The atlantic monthly **176**(1), 101–108 (1945).

[15] S. Su, K. Zhu, E. Xiao, R. Alur, and D. Kang (Bridgewater AIA Labs), "Learning to Replicate Expert Judgment in Financial Tasks," *Thinking Machines Lab: News* (2026), `https://thinkingmachines.ai/news/learning-to-replicate-expert-judgment-in-financial-tasks/`.

[16] R. Sutton, "The Bitter Lesson" (2019), `http://www.incompleteideas.net/IncIdeas/BitterLesson.html`.

[17] `zemax-zos-api`: an agent-facing skill for driving Zemax OpticStudio through the ZOS-API (2026). GitHub repository, `https://github.com/USNavalResearchLaboratory`.

[18] In machine learning, model distillation transfers capabilities from a large model to a smaller one, while execution trajectories refer to the logged sequences of agent prompts, generated API calls, and environment feedback.